Увод в програмирането

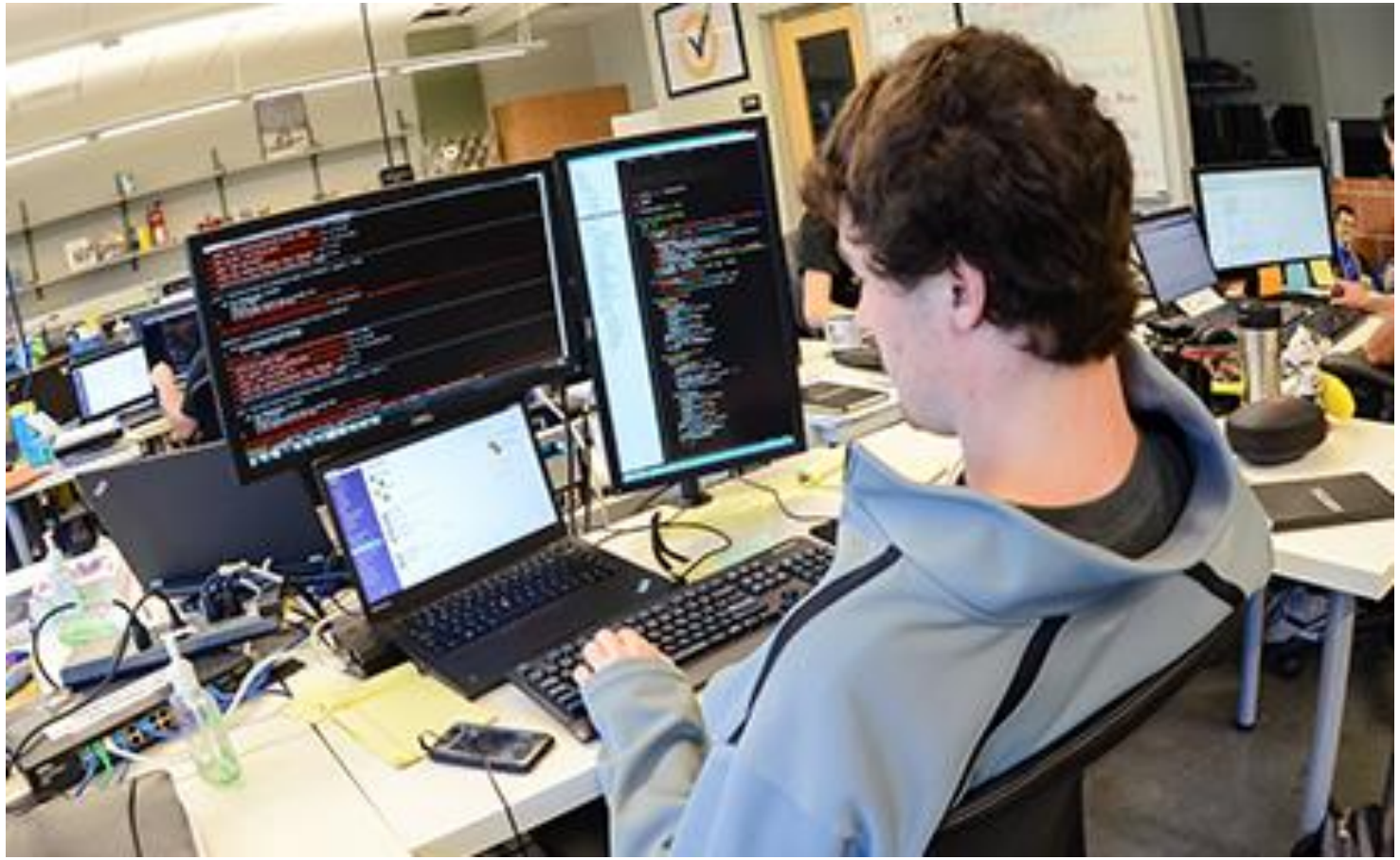
Основи на езика C++

ФМИ, специалност „Софтуерно инженерство“

Съдържание

- Въведение
  - Какво е програмиране. Какво означава софтуер, какво означава програма
  - Понятие за компютър, процесор, памет и променливи
- Елементи на езика
- Основни типове данни.
- Основни аритметични операции; Оператор за присвояване.
- Празен оператор.
- Приоритет на операциите (съкратена версия).
- Вход и изход.

Какво е софтуерно инженерство/програмиране



Софтуер и хардуер

- На български: програмно осигуряване и апаратно осигуряване
- Най-общо казано за работата на компютъра е необходим следният софтуер:
  - Операционни системи;
  - Приложни програми
- За създаването на приложите програми са необходими и т.нар. интегрирани среди за разработка на софтуер
  - Integrated Development Environment (IDE)

Вашите програми

Среди за разработка, виртуални машини и други

Операционна система

Хардуер

Операционни системи (ОС)

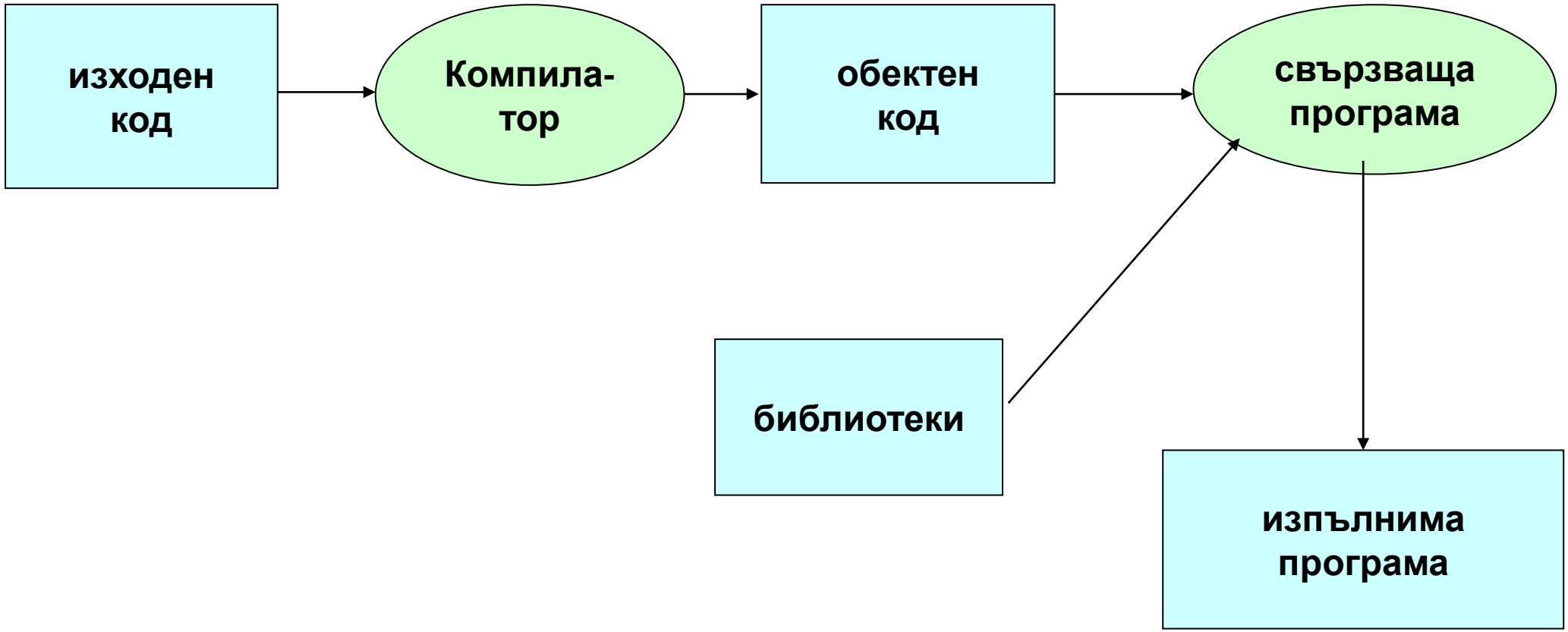
- Всеки компютър работи под управлението на някаква ОС. Тя управлява програмите и:
  - взаимодействието между потребителите и хардуера;
  - периферните устройства (чрез драйвери);
  - файловете (чрез файловата система).
- Широкоразпространени ОС са MS DOS, MS WINDOWS, UNIX, LINUX.
  - Но също и Android, iOS и т.н.

Как се създава една програма

- Какви процеси включва създаването на една компютърна програма
- Но освен това:
  - Проектиране
  - Автоматично създаване на често употребяван код
  - Отстраняване на грешки (debugging)



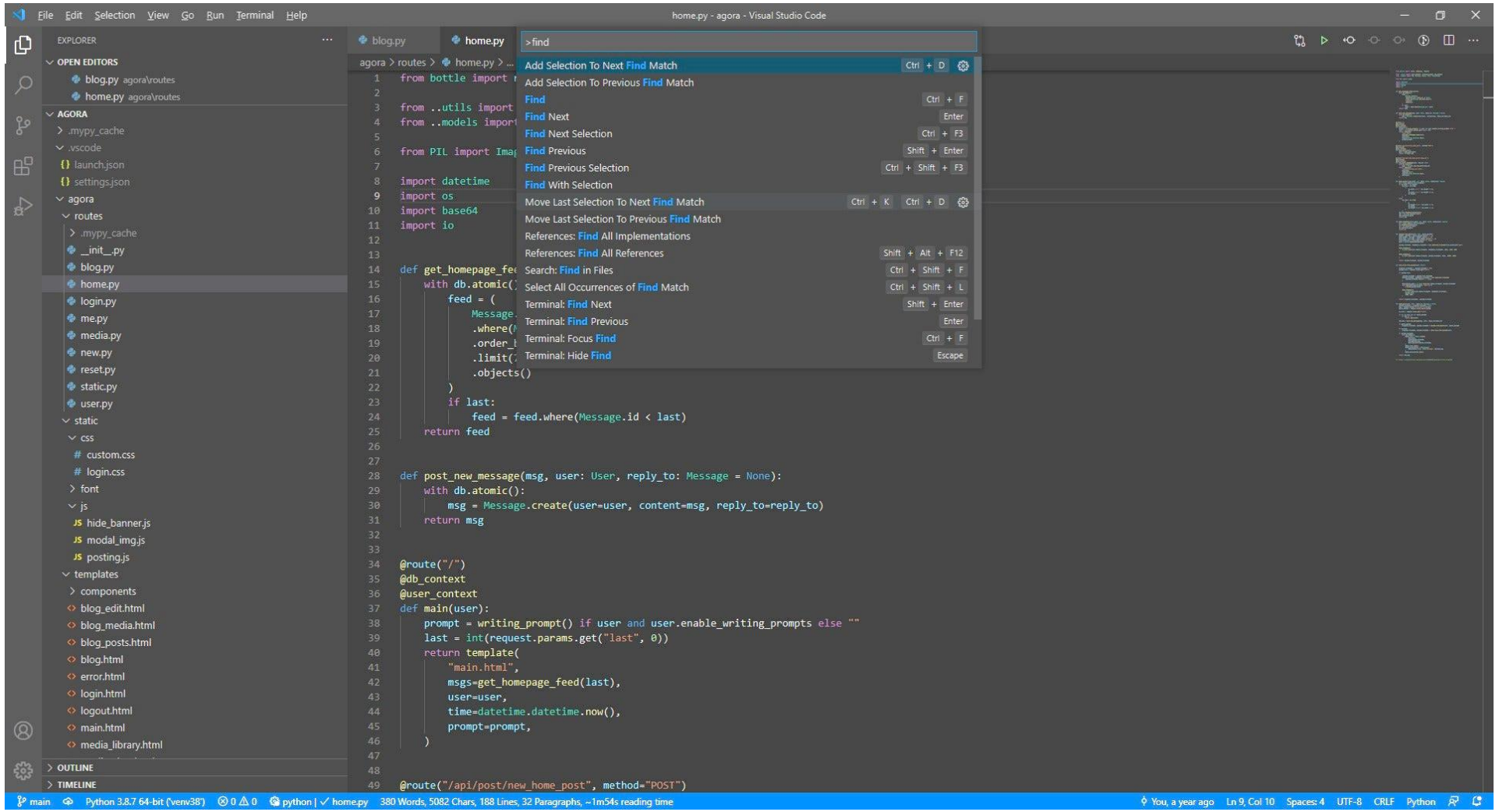
Примерен процес на създаване на програма



Интегрирани среди за разработка

- Целта им е да улеснят процеса на програмиране и да автоматизират някои дейности от него
- Интегрираните среди за разработка включват:
  - Език за програмиране,
  - Транслатор (компилатор или интерпретатор), свързващ редактор,
  - Изпълнителна система,
  - Система за проверка на програмите,
  - Система за поддържане на библиотеки и
  - Текстови редактори.
- Популярни интегрирани среди за разработка
  - Visual Studio
  - Eclipse
  - Code Blocks

Интегрирани среди за разработка (IDE)



Какво представлява компютърът



Процесор

Входно устройство

Памет

Изходно устройство









- Най-общо казано, компютърът е средство за представяне и обработване на информация.
- Компютърът е устройство или система, която може да изпълнява определена последователност от операции
- Това са най-често числови изчисления, аналитични преобразувания или обработка на данни, но включват също и входно/изходни операции.

Първа програма на C++

```
#include <iostream>


using namespace std;

int main()
{
    cout << "Hello world!";

    return 0;
}
```

- Заглавни (header) файлове
- Функция main()
  - Инструкции
  - Оператор return

Структура на програмата на C++

<изходен_файл> ::=

<заглавен_блок_с_коментари>

<заглавни_файлове>

<константи>

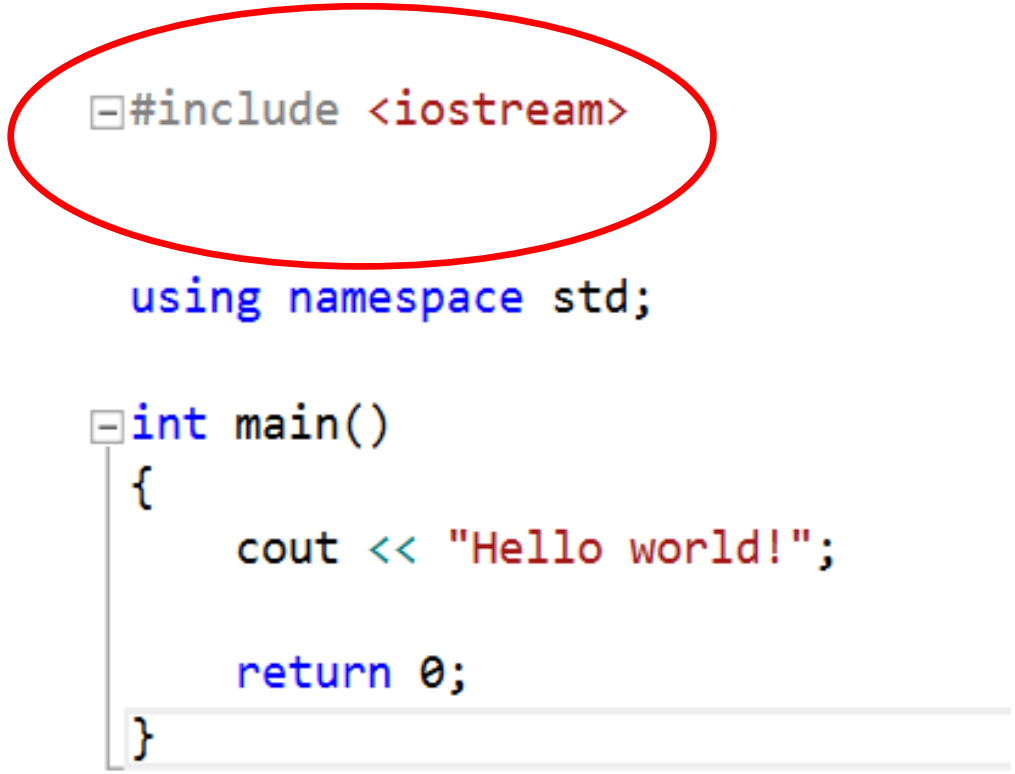
<глобални_променливи>

<класове>

<функции>

Заглавни (header) файлове

- Съдържа декларации, които се използват в повече от една част на програмата.
- Действа като интерфейс между различните компилируеми части на праграмата.
  - Библиотеки
  - Други програми



Азбука на езика C++

- Главните и малки букви на латинската азбука;
- Цифрите
- Специалните символи

+ - * / = ( ) [ ] { } | : ; “ ‘ < > , . _ ! @ # $ % ^ ~

Идентификатор

- Наименование на елемент в програмата
- Поредица от символи
  - Букви, цифри и знака за долна черта
  - Може да започва само със символа ‘$’, буква или долна черта
- Валидни идентификатори:
  - abc, ABC, minValue1, _first_setting, $tree
- Невалидни идентификатори:
  - 1ba, ab+1, a(1)

Лоши практики за имена на идентификатори

```
int $php = 5;
double _$_ = 3;
char nayqkiqsimvolever = 'M';
float MnOgOqKo = 2.4;
bool _ = false;
long __ = 5234;
long long $$ = 42342;
unsigned gjnueirgeri = 453;
```

Добри практики за имена на идентификатори

- Важно е имената да са такива, че да е обясняват ролята на съответния идентификатор
- Не е добре да са много дълги
- Не е препоръчително да са на български с латински букви
  - Някой ден кодът ви може да се чете от колега, който не знае езика
- Има различни препоръки за именоване на идентификатори - т.нар конвенции (naming conventions)

Кое според вас е по-ясно?

- b = b - a
- balance = balance – withdrawn_amount
- balance = balance – withdrawnAmount

Добри практики за имена на идентификатори

```
// Конвенции за именуване на променливите

// Много, много лош стил:
int abfd_77 = x – y998; // 0_o

// Много по-ясно:
int MonthlyProfit = Income – Expenses;

int nMonthlyProfit = fpIncome – fpExpenses;
```

Също добри практики

- int age = 5;
- double sum = 3;
- char coolestSymbol = ‘M’; float MyDegree = 5.6;
- long remaning_slides = 5234;
- bool isOk= true;
- bool hasBlueEyes = false;

Ключови думи

- Вградени в езика идентификатори, които се използват в програмите по стандартен, предварително определен начин
- Ключови думи на C++
  - int, namespace, return и т.н.
- Ключови думи на препроцесора
  - include, undef, ifdef и т.н.
- Ключови думи на компилатора
  - Зависят от конкретния компилатор

Коментари

- Коментарът е произволен текст, ограден със знаците /* и */ или от // и EOL (знак за край на реда).

  <коментар> ::= /* <редица_от_знаци> */ |

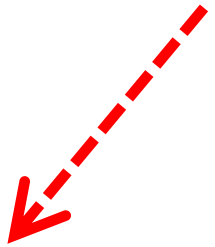
  // <редица_от_знаци> Край на реда

- Пояснява програмен фрагмент. Предназначен е за програмиста. Игнорира се от компилатора на езика.
- Хубаво е всеки файл да започва със заглавен блок с коментари, даващи информация за целта му, за използваните компилатор и операционна система, за името на програмиста и датата на създаването на файла.
- **Писането на коментари значително повишава качеството на кода!!!**

```
/* Това е класическият тип коментар от C */
int x;

/* Той
        може          да бъде
                на няколко реда... */
int y;

int /* ...и да се появи навсякъде */ z;
```



Въпреки, че е възможно, НЕ правете това в своя код!!!

```
// C++-style comments

// Тези коментари завършват в края на реда
int x;

// Ако искате да са на няколко реда
// ще трябва да коментирате всеки от
// от тях поотделно.
int y;
```

```
// По правило коментарите (независимо кой
// стил използвате) се поставят или над това,
// което коментирате…
int x;

int z; // …или вдясно от него.
```

Променливи

- Част от паметта, за съхранение на данни което може да съдържа различни стойности по време на изпълнение на програмата
- Имат три характеристики
  - Тип
  - Име
  - Стойност
- Името на променливата трябва да е валиден идентификатор
- Всяка променлива трябва да има тип, който се указва при дефинирането ѝ

Дефиниране на променливи

<име_на_тип> <променлива> [= <израз>]$_{опц}$

{, <променлива> [= <израз>]$_{опц}$}$_{опц}$;

<име_на_тип> е дума, означаваща име на типа на променливата

<израз> е правило за получаване на стойност – цяла, реална, знакова и друг тип, съвместимо с името на типа на променливата

```
int i = -123, j = 33;

char a = 's';
char b = 115;

double value;

string add;
```

Втора програма на C++

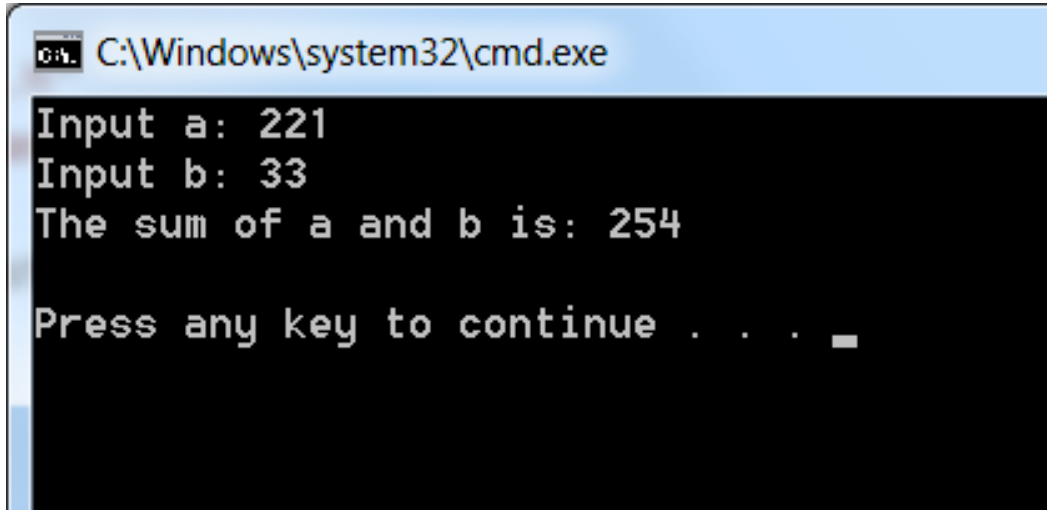
```cpp
#include <iostream>

using namespace std;

int main()
{
    int a = 0, b = 0;

    cout << "Input a: "; cin >> a;
    cout << "Input b: "; cin >> b;
    cout << "The sum of a and b is: " << a + b
         << endl << "\n";

    return 0;
}
```



Основни типове данни

- Скаларни
  - Булев (bool)
  - Символен (char)
  - Целочислен (int)
  - За числа с плаваща запетая (float, double)
- Съставни типове
  - Масив (T[])
    - Символен низ (char [])
  - Структура (struct)
- Още – втората половина на семестъра

Булев (логически) тип - *bool*

- **Множество от стойности**

  Състои се от два елемента – стойностите **true (1)** и **false (0)**
- <булева_константа> ::= true | false
- Основни логически операции
  - && - AND
  - || - OR
  - ! - NOT

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

AND

A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

OR

A	not A
0	1
1	0

NOT

Символен тип - *char*

- Променливите от тип char са с размер 1 байт
- Множество от стойности: [-128; 127]
  - unsigned char: [0, 255]
- Литерали
  - ‘<символ>’,
  - ‘\<контролен_символ>’,
    - ’\n’ – нов ред
    - ‘\t’ – табулация
    - …..

Целочислени типове

- ***short***; множество от стойности – $[-2^{15}; 2^{15}-1]$
- ***int***; множество от стойности – $[-2^{31}; 2^{31}-1]$
- ***long***; множество от стойности – $[-2^{63}; 2^{63}-1]$
- ***long long***; множество от стойности – $[-2^{63}; 2^{63}-1]$

- Числа без знак: ***unsigned***
- ***unsigned short***; множество от стойности – $[0; 2^{16}-1]$
- ***unsigned int***; множество от стойности – $[0; 2^{32}-1]$
- ***unsigned long***; множество от стойности – $[0; 2^{64}-1]$
- и т.н.

Операции с целочислени типове

- Аритметични операции
  - едноместни операции за знак (+, -)
  - двуместни аритметични операции
    - a + b (събиране)
    - a - b (изваждане)
    - a * b (умножение)
    - a / b (целочислено деление - частно)
    - a % b (деление по модул - остатък)
- операции за сравнение (предикати)
  - a == b (равно)
  - a != b (различно)
  - a < b (по-малко)
  - a > b (по-голямо)
  - a <= b (по-малко или равно)
  - a >= b (по-голямо или равно)

Едноместни операции с целочислени типове

- ++
  - събиране с 1
- --
  - изваждане с 1
- Префиксни операции

  --a; ++a
- Постфиксни операции

  a--; a++

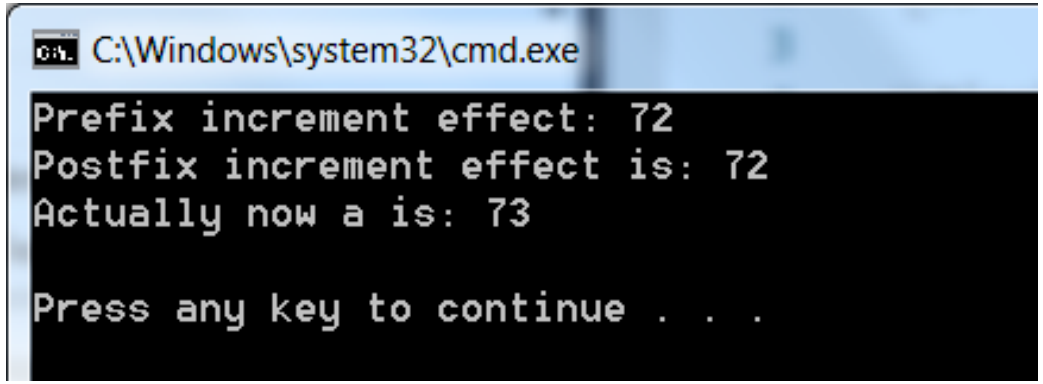
- А каква е разликата между двете?

Префиксни и постфиксни операции

```
int main()
{
    short int a = 71;

    cout << "Prefix increment effect: " << ++a << endl;

    cout << "Postfix increment effect is: " << a++ << endl;
    cout << "Actually now a is: "<< a << endl << endl;
}
```



- Повишената употреба на префиксните и постфиксните операции често води до трудно четим код!!

Представяне на реалните числа

- Какво означава реално число?
  - Колко реални числа има между 0 и 1?
  - А между 0.001 и 0.002?
- В компютъра няма безкрайно количество памет с която да се представят всички реални числа
  - Затова говорим за дробни или още....
- Числа с плаваща запетая
  - Тип float
  - Тип double

Операции с числа с плаваща запетая

- Валидни са всички операции за целочислените типове, без делението по модул
- Резултатът от операцията „/“ е дробно деление, а не частно
- Максималното число:
  - float: ~3,40282e+38
  - double: ~1,79769e+308
- Трябва да се има предвид точността на представянето на реалните числа при сравняването им
- Машинна нула

Преобразуване на типовете

- Явно преобразуване

  <преобразуване> ::= (<тип>)<израз>

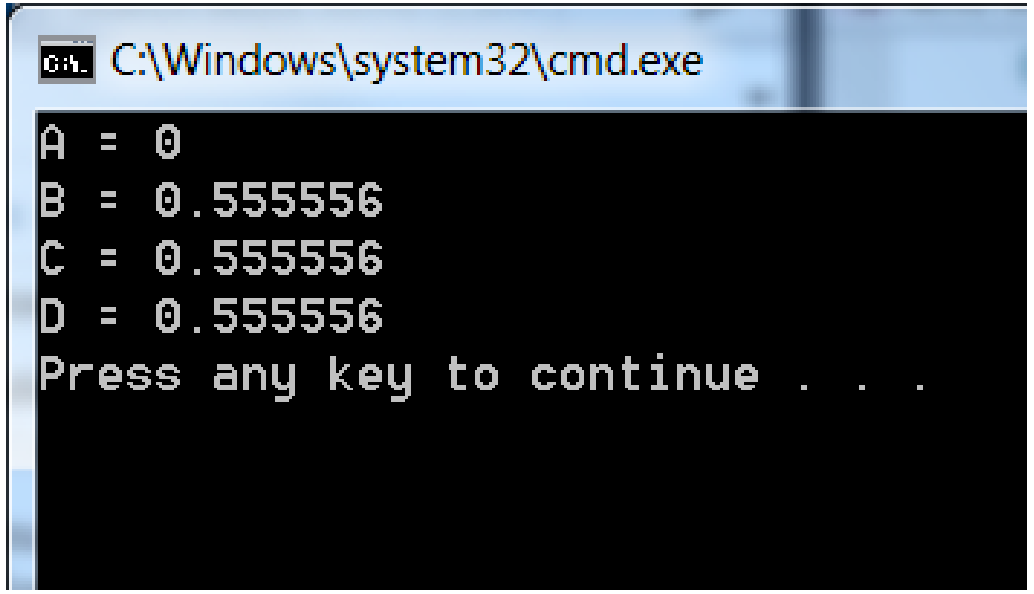
- За да няма загуба на информация се препоръчва преобразуване на типове като се спазва следната посока
  - bool → char → short → int → long → float → double
  - unsigned char → unsigned short → unsigned int → unsigned long

Ефекти от преобразуване на типовете

```
int main()
{
    float A = 5 / 9;
    float B = 5.0 / 9.0;
    float C = 5.0 / 9;
    float D = 5 / 9.0;

    cout << "A = " << A << endl
         << "B = " << B << endl
         << "C = " << C << endl
         << "D = " << D << endl;

    return 0;
}
```



Константи

- Константи
  - const <име_на_тип> <име_на_константа> = <израз>;
  - Добра практика е често повтарящите се стойности да се дефинират като константи, които след това да се използват в програмата

  - Конвенция за именуване на константите е те за се записват само с главни букви

Константи

- Примери

```
const int MAXINT = 32767;
const double RI = 2.5 * MAXINT;
const double PI = 3.14159265;
```

- Предимства
  - Програмите стават по-ясни и четливи.
  - Лесно (само на едно място) се променят стойностите им (ако се налага).
  - Възможността за грешки, възможни при многократното изписване на стойността на константата, намалява.

Литерал

- Константна стойност на променлива, зададена в кода
  - int i=2;
  - char c = ‘a’;
  - bool isReal = true;

  - int i = 023 //число в осмична бройна система
  - int i = 0x1F //число в шеснадесетична бройна система
  - Int I = 0b100011 //число в двоична бройна система

Използване на константи

```
int main()
{
    float a,b,c;
    const float MULTIPLIER = 4.55;

    cout << "Input a "; cin >> a;
    cout << "Input b "; cin >> b;
    cout << "Input c "; cin >> c;

    cout << "First value is "  << a*MULTIPLIER << endl;
    cout << "Second value is " << b*MULTIPLIER << endl;
    cout << "Third value is "  << c*MULTIPLIER << endl;
}
```

Statement (инструкция)

- Команди в дадена компютърна програма, които се изпълняват в определена последователност
- Включват един или повече оператори
- В C++ има няколко вида оператори

Оператор за присвояване на стойност

- <променлива> = <израз>;

  където
  - <променлива> е идентификатор, дефиниран вече като променлива,
  - <израз> е израз от тип, съвместим с типа на <променлива>
- <lvalue> = <rvalue>
  - <lvalue> —място в паметта със стойност, която може да се променя
    - Например – променлива
  - <rvalue> —временна стойност, без специално място в паметта
  - Например: константа, литерал, резултат от пресмятане

Оператор за присвояване на стойност

- Съкратени оператори
  - +=
  - -=
  - *=
  - /=

```
a += 4 ; // Еквивалентно на a = a + 4;
a *= 22; // Еквивалентно на a = a * 22;
// и т.н.
```

Едноместни операции

- `++a` и `--a` връщат `a`, което е lvalue
- `a++` и `a--` връщат rvalue

- `--a += 22; // е валиден израз`
- `a-- += 22; // е невалиден израз`

Съкратени оператори за присвояване

- По-общо:

  e1 op= e2;

  е еквивалентно на

  e1 = (e1) op (e2);

  където op е някой от операторите

  +, -, *, /, %, <<, >>, &, |, ^

Асоциативност на операторите

```
int main()
{
    int a = 2, b = 3;

    cout << (a = b = a + b) << "\n";

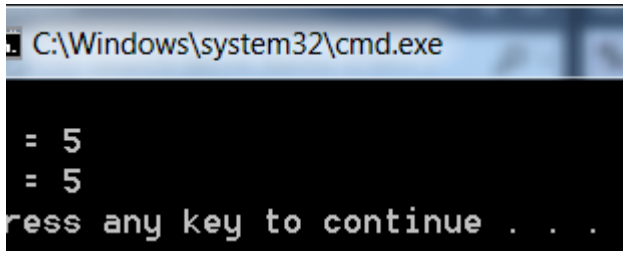
    cout << "a = " << a << endl
         << "b = " << b << endl;


    return 0;
}
```

Операторът за присвояване е дясноасоциативен

a = (b = 2)

<lvalue> = <rvalue>



Асоциативност на операторите

- Освен дясно-, различаваме и ляво- асоциативни оператори
- Например аритметичните оператори са лявоасоциативни
- Какъв ще бъде резултатът от следния израз:

```
double a = 16 / 2 / 2;
```

Приоритет на операторите

Оператор	Име	Асоциативност
++ --	Postfix Increment Postfix Decrement	Left to Right Left to Right
++ --	Prefix Increment Prefix Decrement	Right to Left Right to Left
* /	Multiplication Division	Left to Right Left to Right
+ - %	Addition Subtraction Modulus	Left to Right Left to Right Left to Right
< >	Less Than Greater Than	Left to Right Left to Right
&&	Logical And	Left to Right
||	Logical Or	Left to Right
?:	Conditional	Right to Left
=	Assignment	Right to Left
,	Comma	Left to Right

Приоритет на операторите

- Приоритетът на операторите определя реда, в който те се прилагат в рамките на дадения израз
- Какъв ще бъде резултатът от следния израз:

```
bool isTrue = 30 < 7 + 32/8;
```

Оператор за изброяване

- <израз1>,<израз2>,..., <изразN>
- Оценяват се всички изрази, отляво на дясно (лявоасоциативен)
- Стойността на оператора се определя от най-десния израз
- Използва се рядко

```
int a = 0;

1+2, a, a+10;  // Стойност на израза: 10

a++, a++, a++; // Стойност на израза: 2
               // Стойност на a: 3
```

Празен оператор

- Знакът ;

- Не извършва никакви действия. Използва се когато синтаксисът на някакъв оператор изисква присъствието на поне един оператор, а логиката на програмата не изисква такъв

Съставен оператор (Блок)

- Синтаксис

  {<оператор1>
  <оператор2>
  . . .
  <операторN>
  }

- Семантика
  - Обединява нула или повече оператора в един. Може да бъде поставен навсякъде, където по синтаксис стои оператор
  - Дефинициите в рамките на блока, се отнасят само за него, т.е. не могат да се използват извън него
  - Самият блок не завършва с ;

Области на видимост на променливите

- Нарича се още „област на действие“ (scope)
- Областта на действие се простира от дефиницията на променливата до края на блока, в който е дефинирана
- Дефиниция на променлива със същото име в същия блок е забранена
- Дефиницията на променлива във вложен блок припокрива всички дефиниции със същото име във външните блокове

Област на видимост на променливите

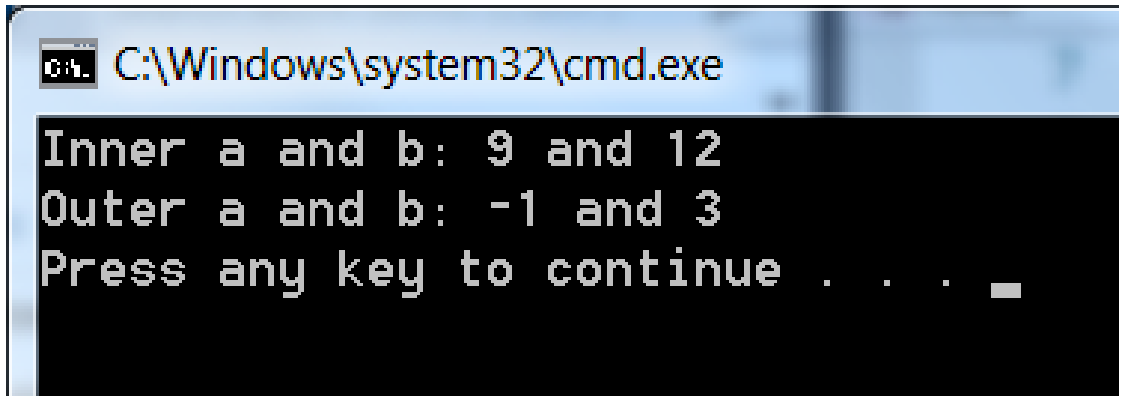
```
int main()
{
    int a = -1;
    int b = 2;

    {
        int a = ++b * 3;
        int b = 12;

        cout << "Inner a and b: " << a
             << " and " << b << endl;
    }

    cout << "Outer a and b: " << a
         << " and " << b << endl;

}
```



Област на видимост на променливите

```
int Variable = 100;

if (Variable > 0)
{
    int Local = 100;
}

cout << "Variable = " << Variable << endl; // OK
cout << "Local = " << Local << endl;       // Грешка!
```

Оператор за вход: „>>“

cin >> <променлива> {>> <променлива>};

където

- cin е обект (променлива) от клас (тип) istream, свързан с клавиатурата,
- <променлива> е идентификатор, дефиниран, като променлива от “допустим тип”, преди оператора за въвеждане. (Типовете int, long, double са допустими).

Извлича (въвежда) от cin (клавиатурата) поредната дума и я прехвърля в аргумента-приемник <променлива>

Оператор за вход: „>>“

Изразът

cin >> <променлива$_1$> >> <променлива$_2$> >> … >> <променлива$_n$>;

е еквивалентен на

cin >> <променлива$_1$>;

cin >> <променлива$_2$>;

…

cin >> <променлива$_n$>;

Оператор за изход: „ <<“

cout << <израз> {<< <израз>};

където

- cout е обект (променлива) от клас (тип) ostream, свързан с екрана на компютъра,
- <израз> е израз от допустим тип. Представата за израз продължава да бъде тази от математиката. Допустими типове са bool, int, short, long, double, float и др.

Операторът << изпраща (извежда) към (върху) cout (екрана на компютъра) стойността на <израз>.

Оператор за изход: „ <<“

Изразът

cout << <израз$_1$> << <израз$_2$> << … << <израз$_n$>;

е еквивалентен на

cout << <израз$_1$>;

cout << <израз$_2$>;

…

cout << <израз$_n$>;

- За подготовката на тази презентация са използвани слайдове на:
  - Доц. Александър Григоров
  - Доц. Атанас Семерджиев
  - Доц. Трифон Трифонов
  - Мартин Илиев